

# Récurtivité dans les algorithmes

[rouanehm@gmail.com](mailto:rouanehm@gmail.com)



# Récursion (Récursivité)

- Types de récursion
  - Récursion fonctionnelle (ou numérique)
  - Récursion structurelle
- Objet récursif :
  - Un objet défini à partir de lui-même.
- Fonction récursive
  - Une fonction est récursive si elle peut s'appeler elle-même de façon directe ou indirecte.
  - Il y a une forte analogie avec les maths :  $(n + 1)! = (n + 1) \times n!$

# Récursion structurelle: Un exemple

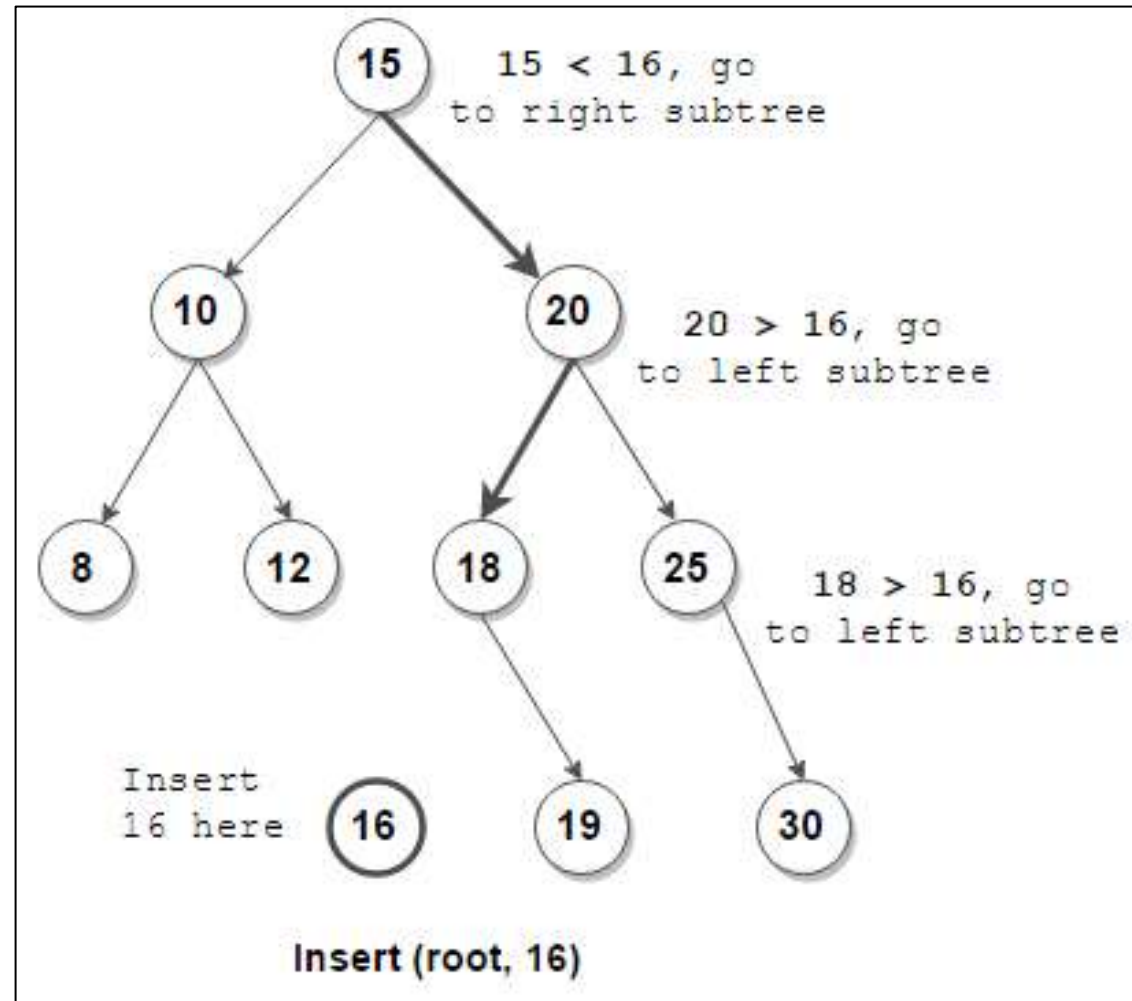
```
// Binary search tree implementation in Java
// Author: AlgorithmTutor

// data structure that represents a node in the tree
class Node {
    int data; // holds the key
    Node parent; // pointer to the parent
    Node left; // pointer to left child
    Node right; // pointer to right child

    public Node(int data) {
        this.data = data;
        this.parent = null;
        this.left = null;
        this.right = null;
    }
}

public class BST {
    private Node root;

    public BST() {
        root = null;
    }
}
```



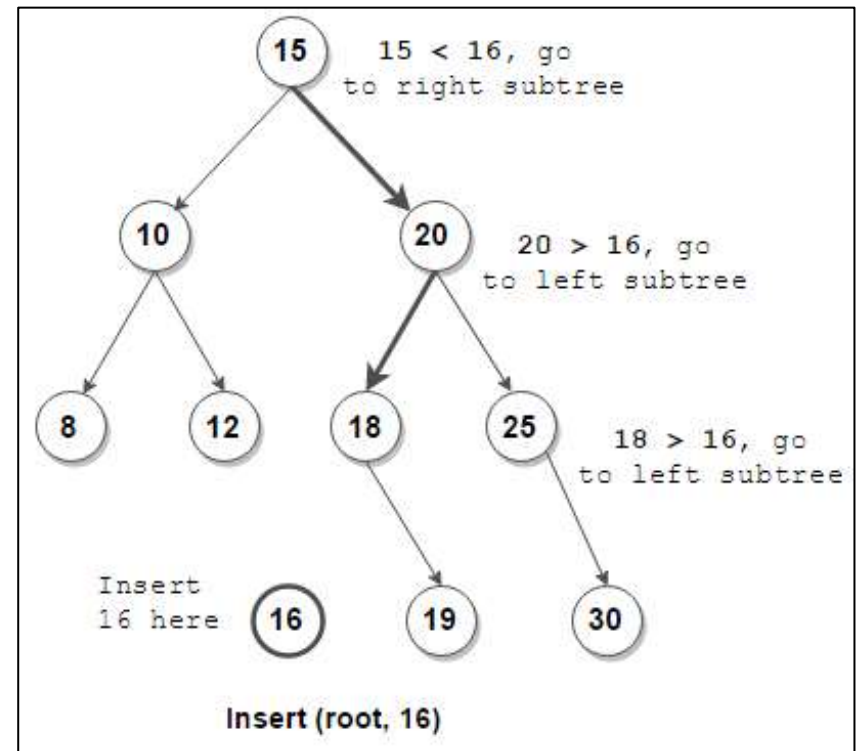
# Récursion fonctionnelle:

## Un exemple

```
1  # Data structure to store a Binary Search Tree node
2  class Node:
3      def __init__(self, data, left=None, right=None):
4          self.data = data
5          self.left = left
6          self.right = right
7
8
9  # Recursive function to insert a key into BST
10 def insert(root, key):
11
12     # if the root is None, create a node and return it
13     if root is None:
14         return Node(key)
15
16     # if given key is less than the root node, recur for left subtree
17     if key < root.data:
18         root.left = insert(root.left, key)
19
20     # if given key is more than the root node, recur for right subtree
21     else:
22         root.right = insert(root.right, key)
23
24     return root
```

<https://www.techiedelight.com/determine-given-binary-tree-is-a-bst-or-not/>

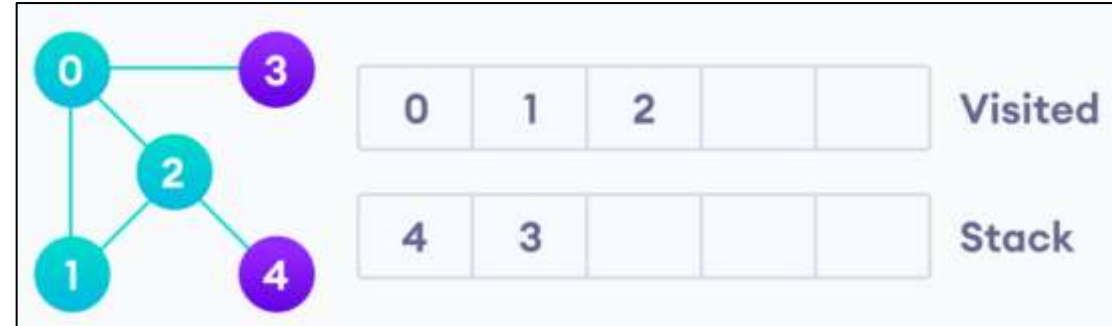
```
61 if __name__ == '__main__':
62
63     root = None
64     keys = [15, 10, 20, 8, 12, 16, 25]
65
66     for key in keys:
67         root = insert(root, key)
```



<https://www.techiedelight.com/insertion-in-bst/>

# Utilité de la récursion

- Certains problèmes en informatique sont de nature récursive
  - Ne peuvent pas être résolus sans récursion
  - Ex.: définir le concept de 'ancêtre'
    - Les parents sont des ancêtres
    - Les ancêtres des parents sont des ancêtres
- Certains problèmes peuvent être formulés de façon plus concise et claire avec récursivité



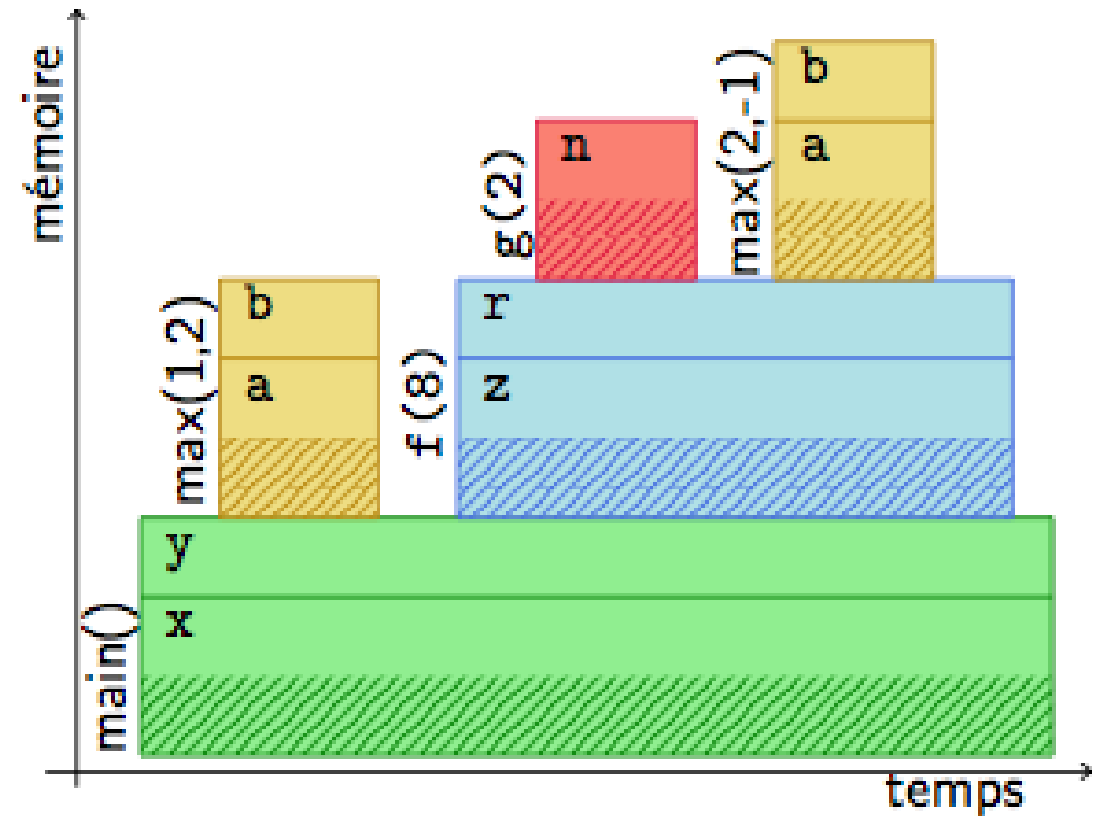
```
DFS(G, u)
    u.visited = true
    for each v ∈ G.Adj[u]
        if v.visited == false
            DFS(G,v)

init() {
    For each u ∈ G
        u.visited = false
    For each u ∈ G
        DFS(G, u)
}
```

# Traitement de la récursivité fonctionnelle :

## La pile d'exécution (1)

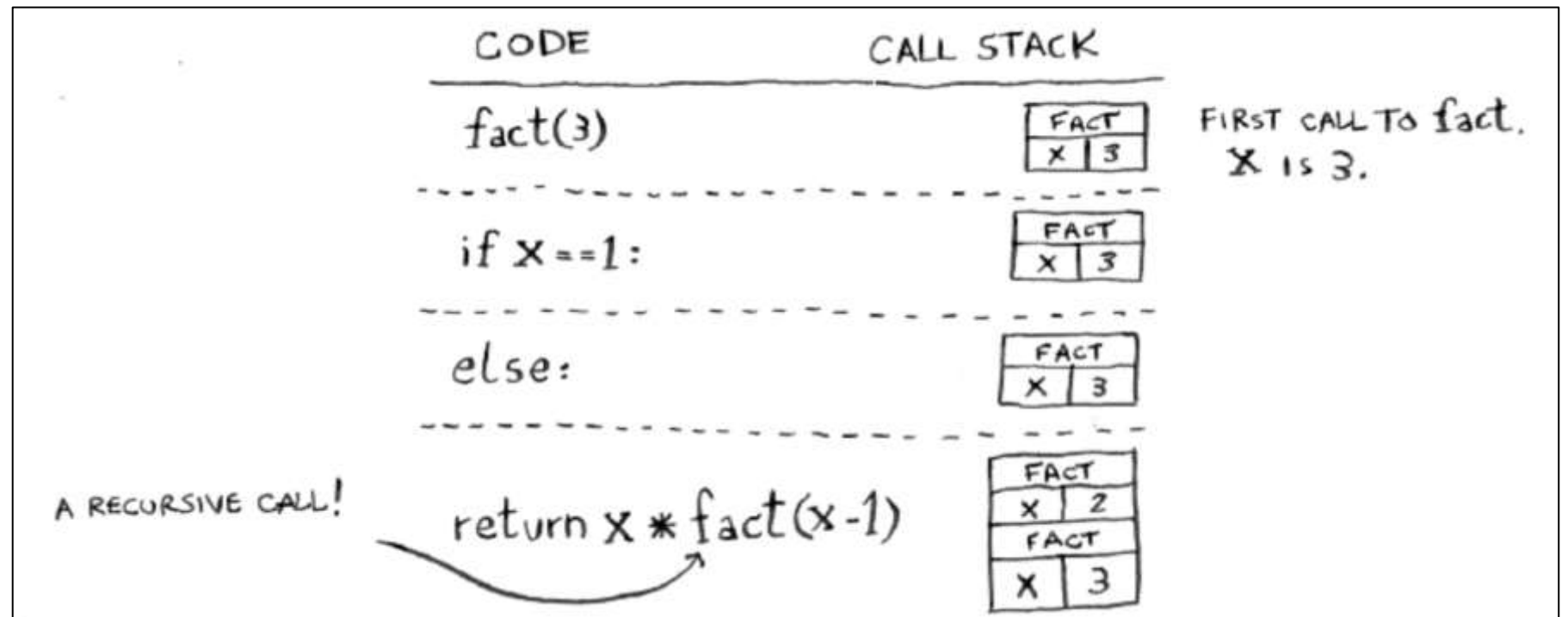
- Les piles sont des structures qui peuvent contenir plusieurs éléments. Toutefois, seul le dernier élément ajouté à la pile est accessible.
- Lorsqu'une fonction est appelée, ses paramètres formels, l'adresse de retour et les variables locales (automatiques) de la fonction appelée sont déposés sur la pile. Toutes ces données sont retirées de la pile à la fin de l'exécution de la fonction. Conséquemment, lors d'appels récursifs, les variables de la fonction sont empilées en cascade.



# Traitement de la récursivité fonctionnelle : La pile d'exécution (2)

```
1 def fact(n):  
2     if n == 1:  
3         return n  
4     else:  
5         return n * fact(n-1)
```

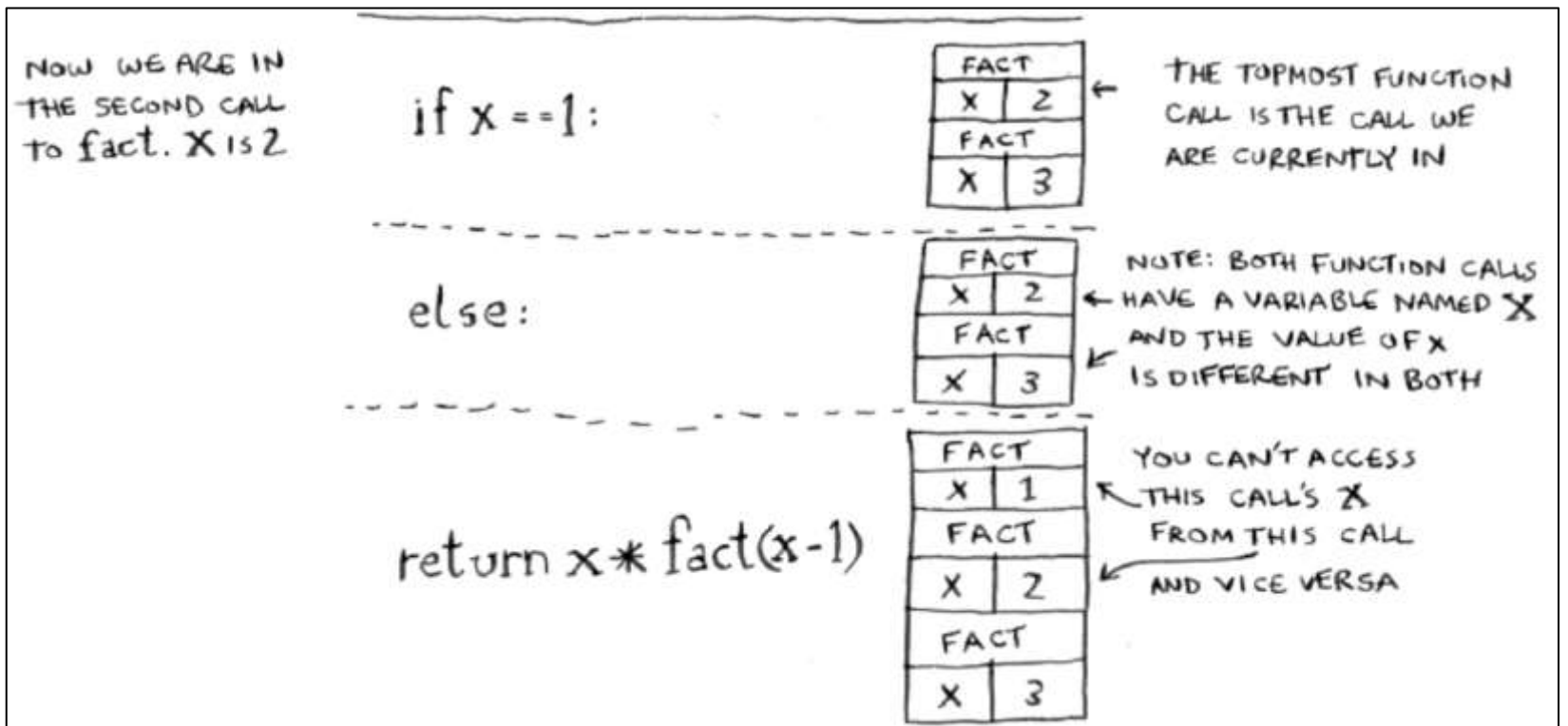
<https://www.freecodecamp.org/news/how-recursion-works-explained-with-flowcharts-and-a-video-de61f40cb7f9/>



# Traitement de la récursivité fonctionnelle : La pile d'exécution (2)

```
1 def fact(n):  
2     if n == 1:  
3         return n  
4     else:  
5         return n * fact(n-1)
```

<https://www.freecodecamp.org/news/how-recursion-works-explained-with-flowcharts-and-a-video-de61f40cb7f9/>

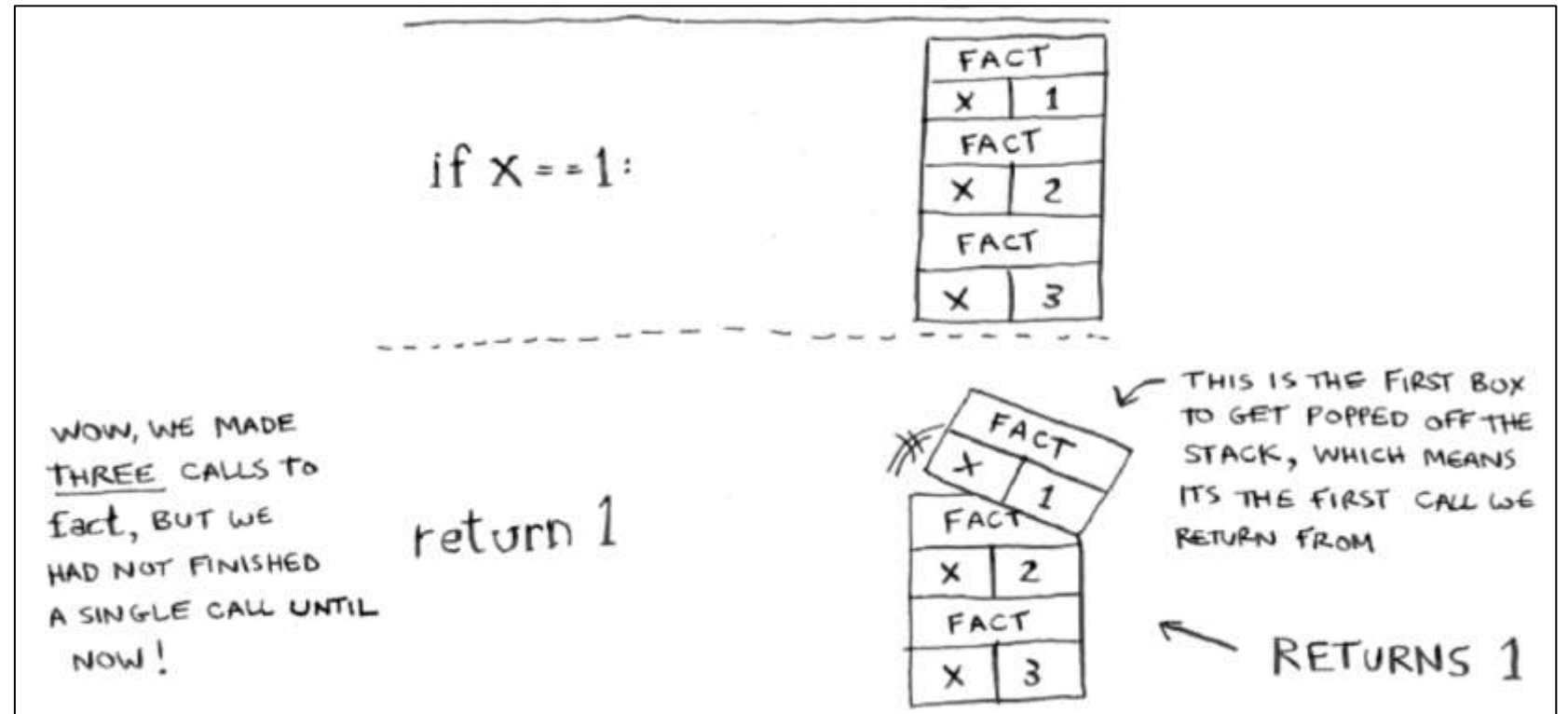


# Traitement de la récursivité fonctionnelle :

## La pile d'exécution (2)

```
1 def fact(n):  
2     if n == 1:  
3         return n  
4     else:  
5         return n * fact(n-1)
```

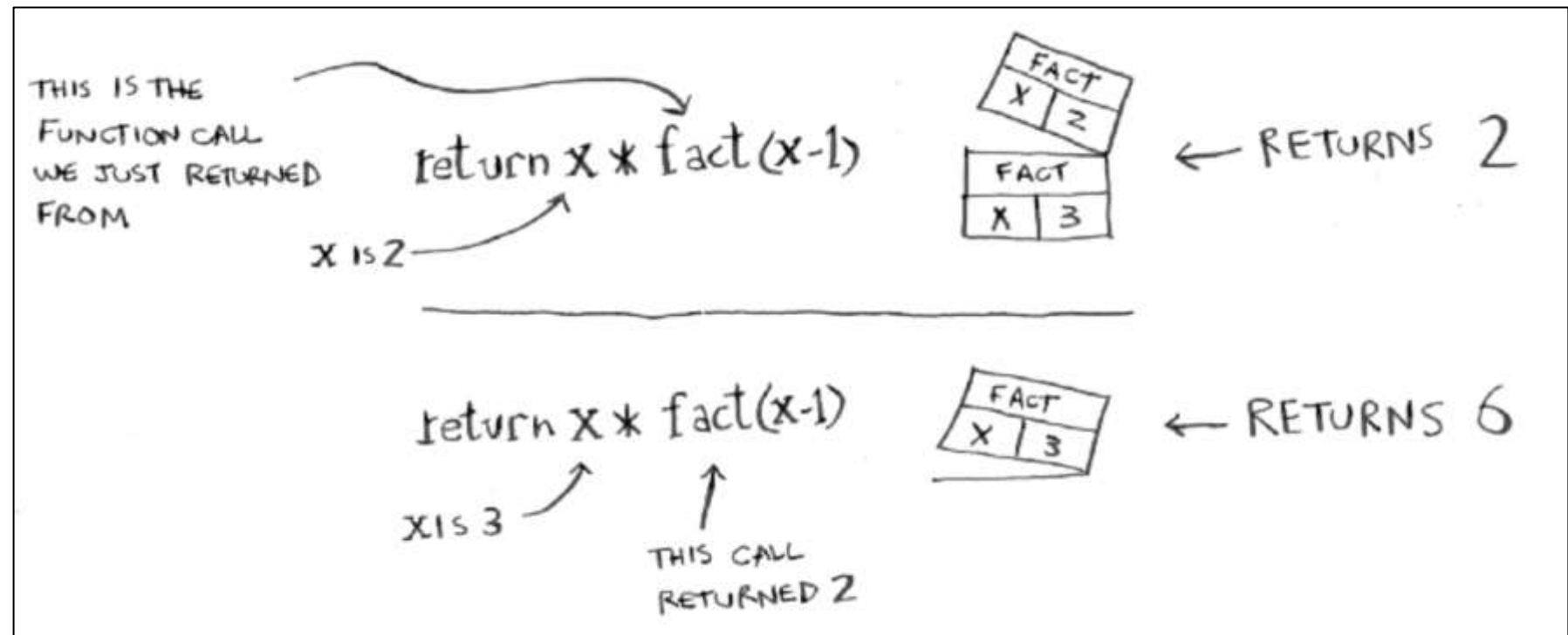
<https://www.freecodecamp.org/news/how-recursion-works-explained-with-flowcharts-and-a-video-de61f40cb7f9/>



# Traitement de la récursivité fonctionnelle : La pile d'exécution (2)

```
1 def fact(n):  
2     if n == 1:  
3         return n  
4     else:  
5         return n * fact(n-1)
```

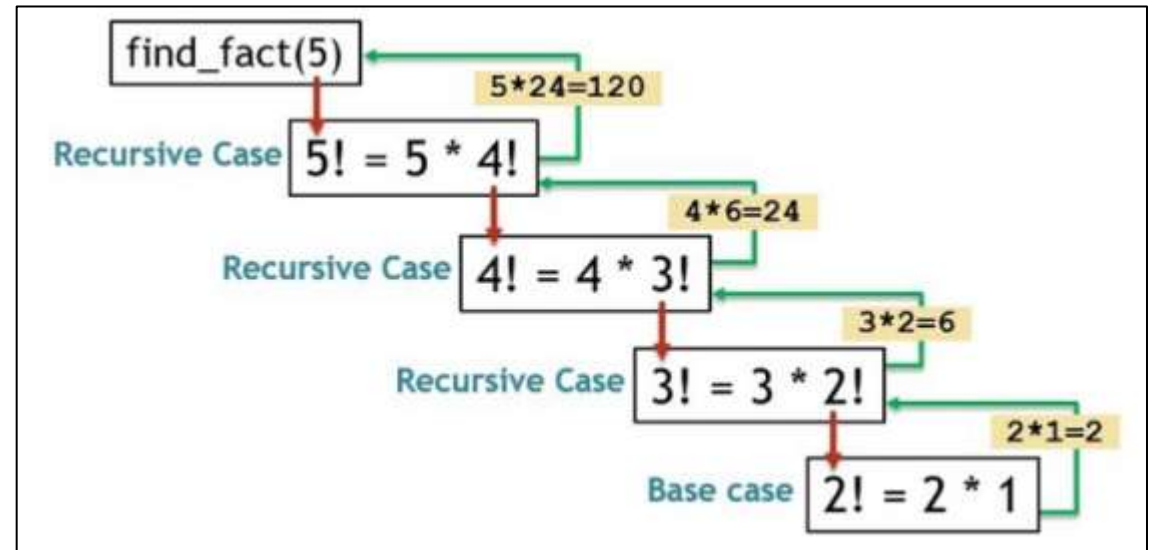
<https://www.freecodecamp.org/news/how-recursion-works-explained-with-flowcharts-and-a-video-de61f40cb7f9/>



# Principes de l'approche récursive (1)

- La fonction est partiellement définie par elle-même
- Décomposition du problème :
  - On connaît certaines réponses (simples)
    - Ex.:  $0! = 1$
  - On sait comment décomposer un cas complexe en cas moins complexe
    - Ex.:  $n! = n * (n-1) !$

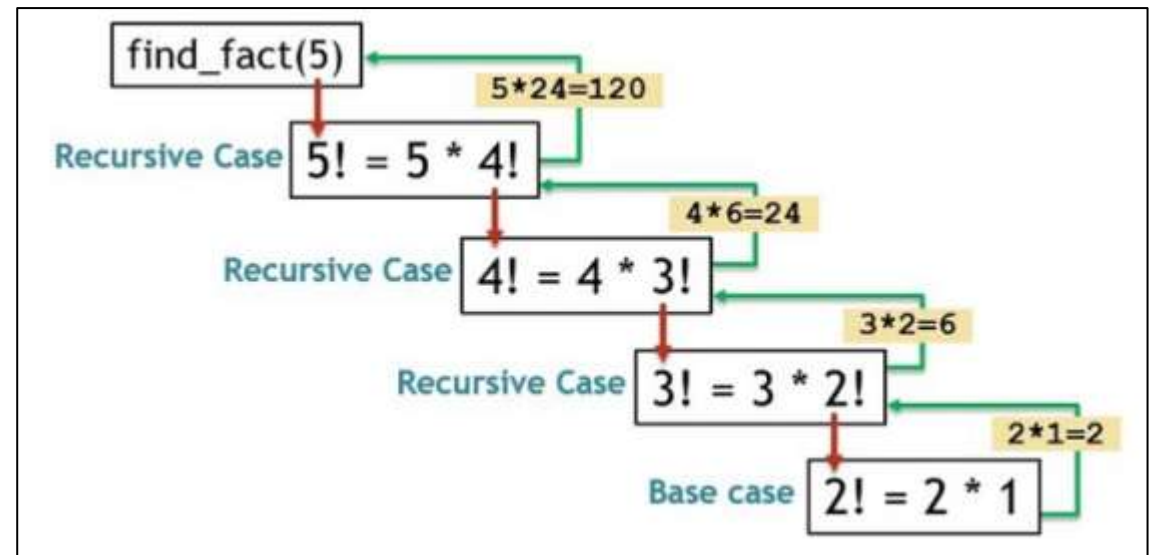
```
1  def fact(n):  
2      if n == 1:  
3          return n  
4      else:  
5          return n * fact(n-1)
```



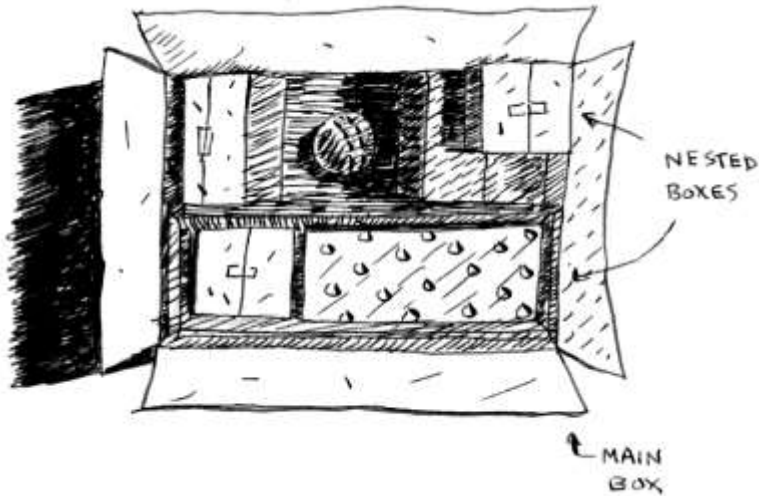
# Principes de l'approche récursive (2)

- La réalisation d'une tâche par un algorithme récursif repose sur deux éléments :
  - La résolution partielle du problème d'une façon simple.
    - Ex.  $0! = 1$
  - La réalisation du reste de la tâche étant déléguée aux appels récursifs successifs.
    - Ex.  $n! = n * (n-1)!$
  - La détermination d'une condition d'arrêt qui permet d'arrêter la cascade d'appels récursifs.
    - Ex.  $\text{if}(n=1) n!=1$

```
1  def fact(n):  
2      if n == 1:  
3          return n  
4      else:  
5          return n * fact(n-1)
```

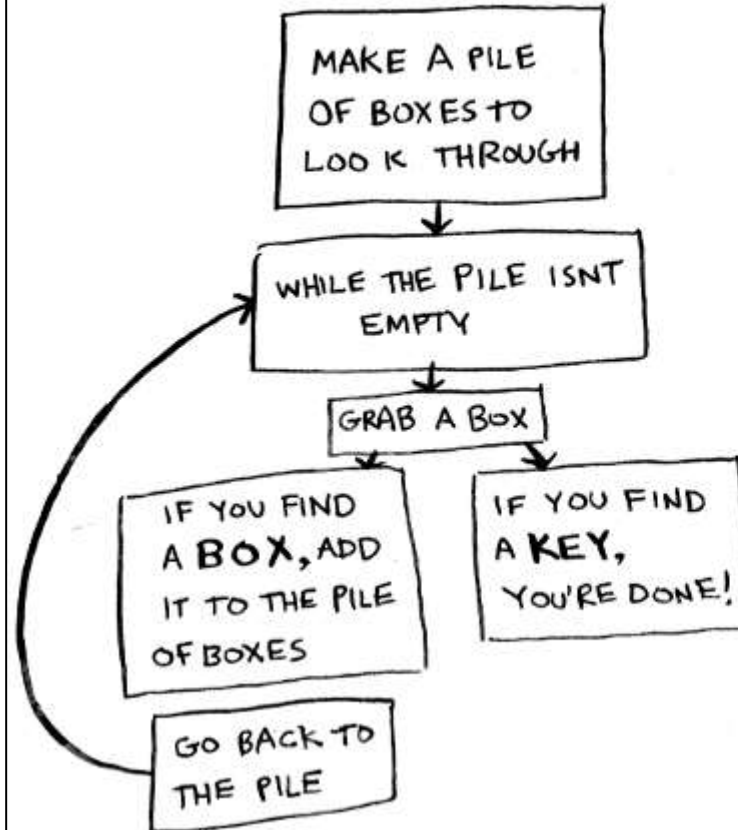


# Principes de l'approche récursive (3): Illustration des principes

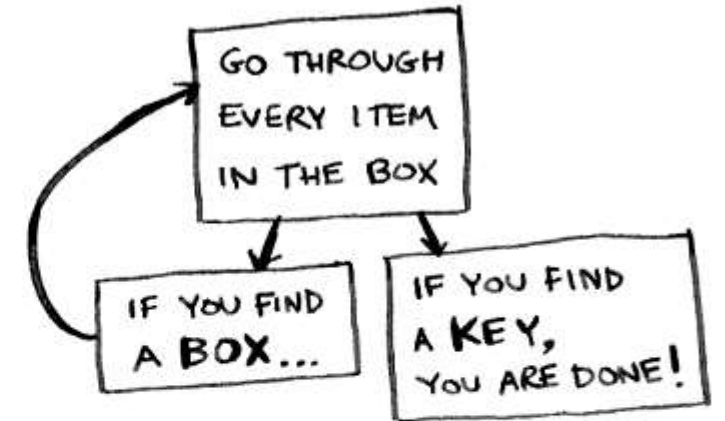


<https://www.freecodecamp.org/news/how-recursion-works-explained-with-flowcharts-and-a-video-de61f40cb7f9/>

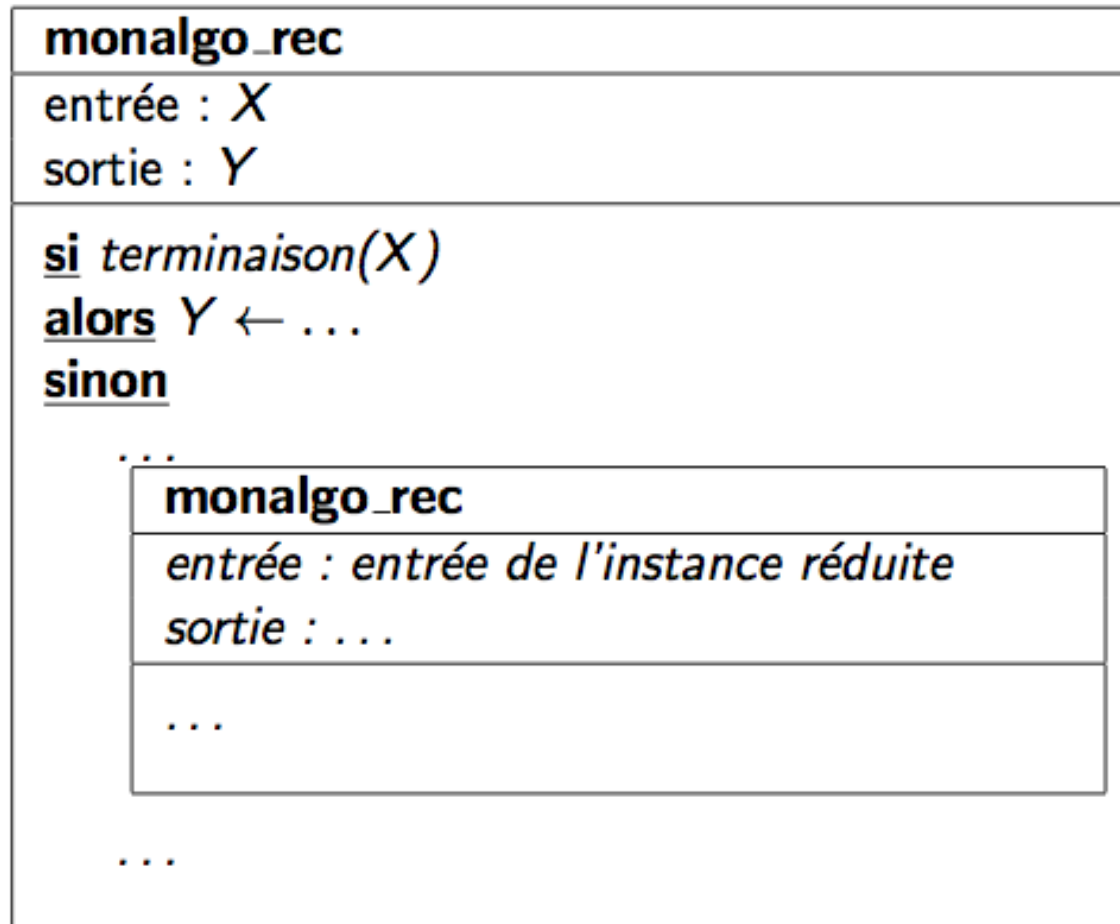
## Iterative Approach



## Recursive Approach



# Schéma général d'un algorithme récursif



# Quelques applications de la récursion

- La plupart des problèmes (ex. le tri fusion d'un tableau), le calcul d'une équation (ex. la taille d'un dossier sur le disque), etc., peuvent être résolus de façon récursive
- Structures de données définies récursivement (listes, arbres, graphes, etc.)
- Équations de récurrence (algébriques, vectorielles, booléennes, formelles, ensemblistes, etc.);

```
def merge_sort(values):  
    if len(values)>1:  
        m = len(values)//2  
        left = values[:m]  
        right = values[m:]  
        left = merge_sort(left)  
        right = merge_sort(right)  
  
        values = []  
  
        while len(left)>0 and len(right)>0:  
            if left[0]<right[0]:  
                values.append(left[0])  
                left.pop(0)  
            else:  
                values.append(right[0])  
                right.pop(0)  
  
        for i in left:  
            values.append(i)  
        for i in right:  
            values.append(i)  
  
    return values
```

# Avantages et inconvénients de la récursivité

- Avantages:
  - Formulation compacte, claire et élégante.
  - Maîtrise des problèmes dont la nature même est récursive.
- Inconvénients:
  - Possibilité de grande occupation de la mémoire.
  - Temps d'exécution peut être plus long.
  - Estimation difficile de la profondeur maximale de la récursivité.

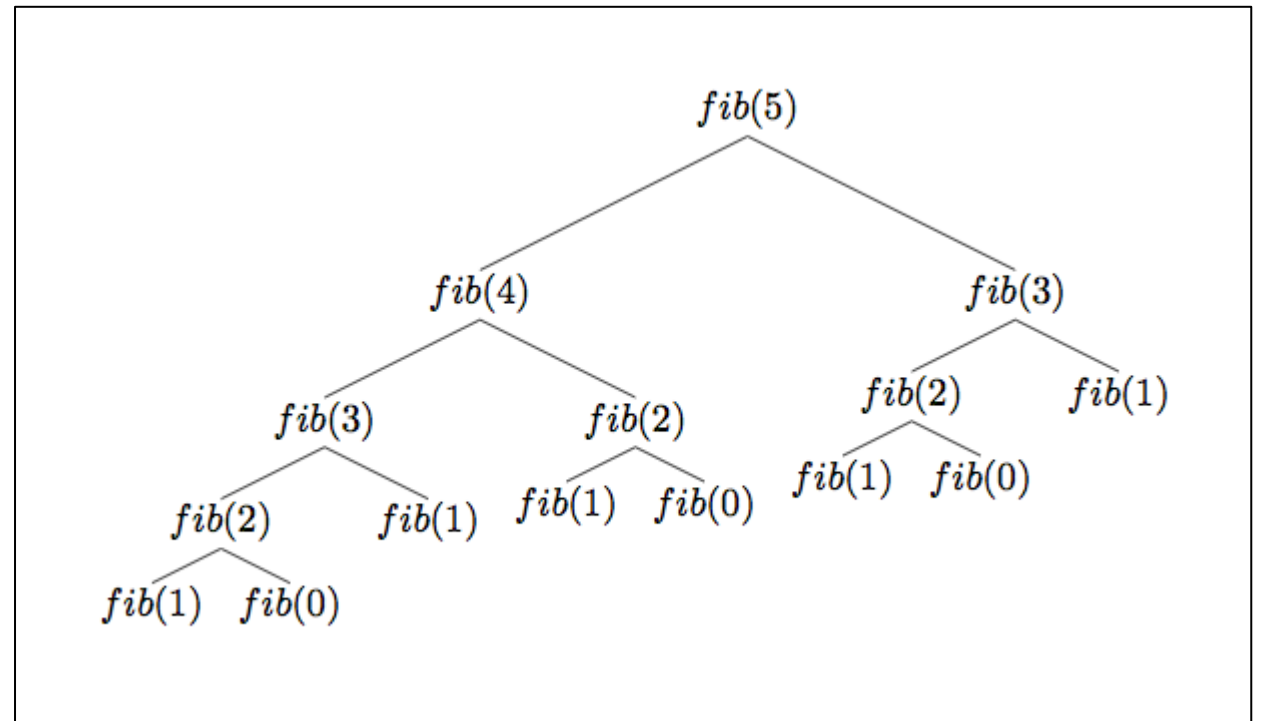
# Exemples de récursivité:

## Suite de Fibonacci

- La suite de Fibonacci est une suite de nombres entiers dans laquelle chaque terme est la somme des deux termes qui le précèdent
- Une fonction de nature récursive car la valeur de  $\text{fib}(n)$  dépend de  $\text{fib}(n-1)$  et  $\text{fib}(n-2)$

## Suite de Fibonacci

0 1 1 2 3 5 8 13 21 34



# Exemples de récursivité:

## Suite de Fibonacci

- Cas simple
  - Problème directement solvable
  - Pas de récursion
- Cas récursif (cas complexe)
  - Appel à la même fonction pour des cas plus simples
  - $n-1$ ,  $n-2$  sont plus simples que  $n$

```
8 def fib(n):
9     if n<2:
10         return n
11     return fib(n-1) + fib(n-2)
12
13 x = [ fib(i) for i in range( 35 ) ]
14 print(x)
```

$$fibo(n) = \begin{cases} 1, & n = 0 \\ 1, & n = 1 \\ fibo(n-1) + fibo(n-2), & n > 1 \end{cases}$$

} Cas simples

Cas récursif

# Exemples de récursivité:

## Suite de Fibonacci (récursion versus itération)

- Pour fib(46), la solution récursive prend 19 secondes, alors que le même résultat est obtenu en moins d'une milliseconde avec l'implémentation itérative ! (AMD Athlon64 3000+).
- Cette fonction récursive effectue plusieurs appels au même calcul. Par exemple pour déterminer  $f(n)$ , on calcule d'abord  $f(n-1)$ , puis au retour de l'appel,  $f(n-2)$  est calculé. Or, dans le calcul de  $f(n-1)$ ,  $f(n-2)$  est déjà calculé.
- **Programmation dynamique:** ne jamais dupliquer le travail par la résolution d'une même instance d'un problème dans plusieurs appels récursifs.

```
1  def fib(number_of_terms):
2      counter = 0
3
4      first = 0
5      second = 1
6      temp = 0
7
8      while counter <= number_of_terms:
9          print(first)
10         temp = first + second
11         first = second
12         second = temp
13         counter = counter + 1
14
15     # Driver Code
16     fib(10)
```

# Exemples de récursivité:

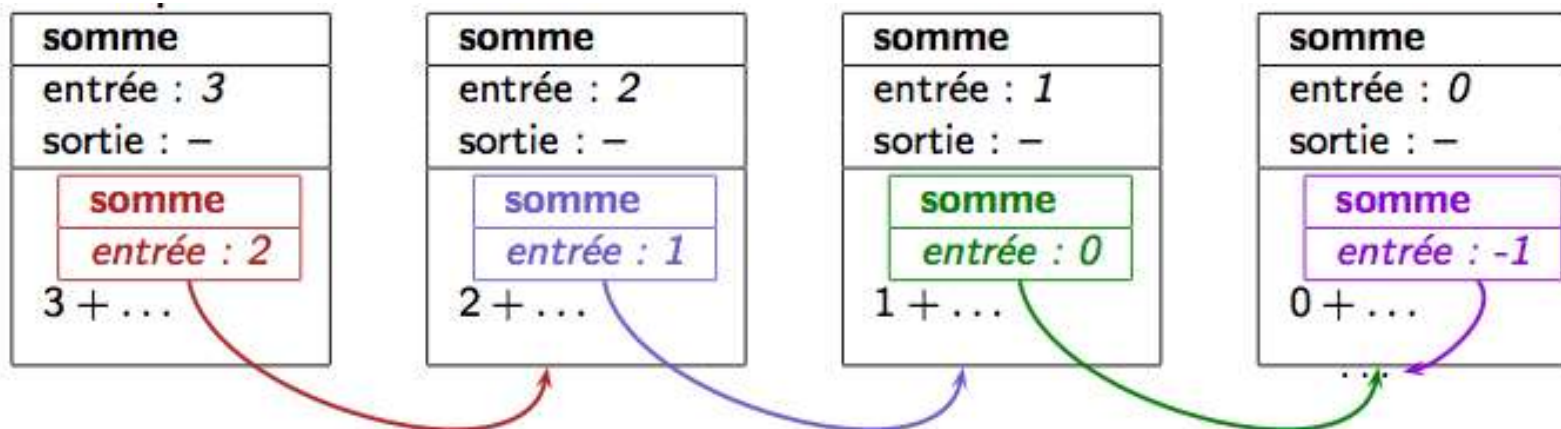
## Somme de n premiers entiers positifs

- Calculer la somme des n premiers entiers : si je sais le faire pour n, je sais le faire pour n+1 :
  - $S(n + 1) = (n + 1) + S(n)$

```
# Python code to find sum
# of natural numbers upto
# n using recursion

# Returns sum of first
# n natural numbers
def recurSum(n):
    if n <= 1:
        return n
    return n + recurSum(n - 1)

# Driver code
n = 5
print(recurSum(n))
```



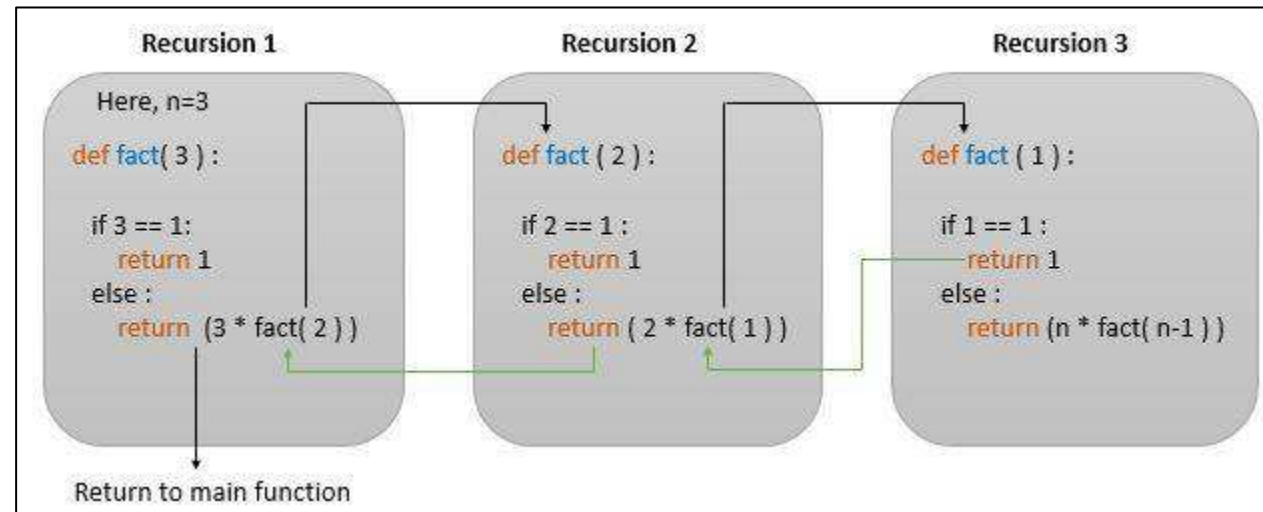
# Exemples de récursivité:

## Factorielle d'un entier positif

$$f : \mathbb{N} \rightarrow \mathbb{N}$$

$$n \mapsto \begin{cases} n \times f(n-1) & \text{si } n > 0 \\ 1 & \text{sinon} \end{cases}$$

```
1  def fact(n):  
2      if n == 1:  
3          return n  
4      else:  
5          return n * fact(n-1)
```



# Exemples de récursivité:

## Parcours d'un tableau (1/2)

- Problème initial: traverser le tableau à partir de l'indexe 0 (jusqu'à la fin)
- Le problème est décomposable ?
  - Pour traverser à partir de l'indexe  $i$ 
    - Traverser l'élément courant  $i$
    - Traverser le reste du tableau – à partir de  $i+1$
  - Attention: condition d'arrêt!
    - Traverser le reste – à partir de  $i+1$  **(si il en reste)**
- Structure de la méthode `traverser(i, tableau)`
  - Traiter l'élément à la position  $i$
  - Si non à la fin du tab, `traverser(i+1, tableau)`

# Exemples de récursivité:

## Parcours d'un tableau (2/2)

- Deux types de parcours :
  - Croissant
  - Decroissant

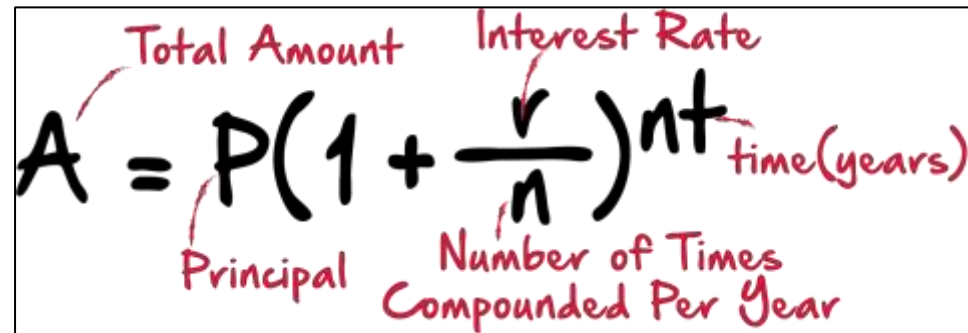
```
2      # Developer : Amine - 2020-09-03 03:20 PM
3      def ascending_order_traversal(values, index):
4          if 0 <= index < len(values):
5              process(values[index])
6              ascending_order_traversal(values, index + 1)
7
8
9      def descending_order_traversal(values, index):
10         if 0 <= index < len(values):
11             descending_order_traversal(values, index + 1)
12             process(values[index])
```

```
15     # Developer : Amine - 2020-09-03 03:25 PM
16     def process(value):
17         print(value)
18
19
20     if __name__ == '__main__':
21         data = ["apple", "orange", "mango", "grapes"]
22         # ascending_order_traversal(data, 0)
23         descending_order_traversal(data, 0)
```

# Exemples de récursivité:

## Calcul du taux d'intérêt composé

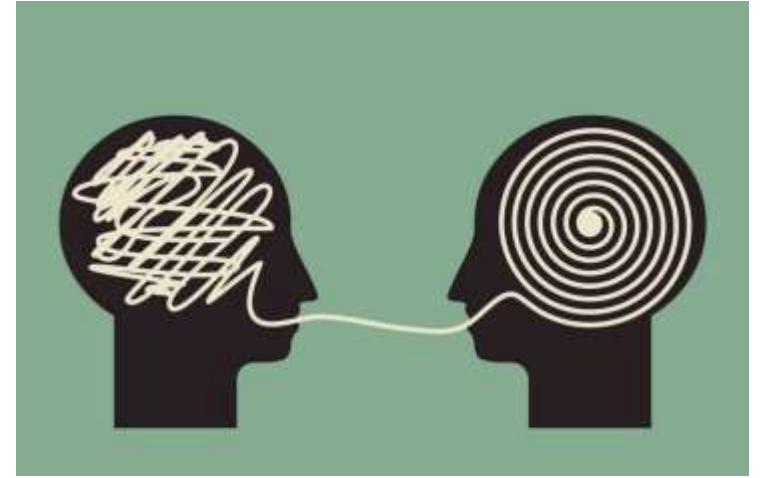
- Dit composé car l'intérêt est calculé en fonction du montant initial en plus des intérêts accumulés à la fin de chaque période de capitalisation.
- A: capital cumulé
- P: capital initial investi
- r: taux intérêt
- n: nombre de périodes de capitalisation par année
- t: nombre d'années d'investissement
- Codez cet exemple



A handwritten formula for compound interest is shown within a rectangular box. The formula is  $A = P(1 + \frac{r}{n})^{nt}$ . Red handwritten labels with arrows point to the variables: 'Total Amount' points to A, 'Interest Rate' points to r, 'Principal' points to P, 'Number of Times Compounded Per Year' points to n, and 'time(years)' points to t.

$$A = P(1 + \frac{r}{n})^{nt}$$

# Récursion versus itération :



- La récursivité est légèrement moins rapide qu'un algorithme itératif équivalent
  - Temps nécessaire à l'empilage et au dépilage des données
  - Les résultats intermédiaires sont gardés sur la pile d'exécution
  - Ex.: Factoriel est mieux implanté itérativement
- La récursivité est plus « élégante »
- Les algorithmes récursifs sont souvent plus faciles à écrire

# Références

- <https://www.techiedelight.com/list-of-problems/>
- <https://algorithmtutor.com/Data-Structures/Tree/Binary-Search-Trees/>
- <https://hackernoon.com/recursion-vs-looping-in-python-9261442f70a5>